

Hybrid R-Sentry: Multi-Surface Behavioral Detection and Kernel-Enforced Containment for Early Ransomware Defense on Linux Endpoints

Mohammad Hudib, Adnan Alshrouqi, Ahmad Jihad, Mahmoud Hussein

Department of Cybersecurity, Philadelphia University

mohammadhudib960@gmail.com, adnan.alshrouqi@gmail.com, ahmad.j.abuzarqa@gmail.com, mahmoudahmad.ma5@gmail.com

Abstract—Modern crypto-ransomware can fully encrypt a user corpus in under two seconds, faster than any reactive defense can respond, and sophisticated variants are now capable of recognizing and skipping static decoy files entirely. This paper presents Hybrid R-Sentry, a host-based ransomware detection and auto-containment system for Linux endpoints that defeats both speed and evasion by combining five independent behavioral detection surfaces with kernel-level enforcement and an AI-powered threat analyst. Building on the R-Sentry deception model, the system adds four complementary layers beyond canary detection: rename and extension analysis, rolling Shannon entropy-velocity profiling, non-sequential write-offset detection for silent in-place encryption, and process lineage scoring with cryptographic binary verification against the dpkg integrity database. Canary placement itself is not static; a Markov chain repositioner continuously predicts the next directories ransomware is likely to traverse and moves decoys there before the attacker arrives. Where possible, detection is enforced directly inside the kernel: an eBPF sensor instruments five syscalls (`openat`, `vfs_write`, `unlink`, `rename`, `execve`), maintains a per-process behavioral score in a BPF hash map, and blocks confirmed threats inline via BPF-LSM before a single byte of user data is overwritten. When a CRITICAL threshold is reached, a tree-aware containment pipeline freezes the entire malicious process tree with SIGSTOP, captures forensic evidence, isolates outbound network traffic at the cgroup level, and terminates with SIGKILL, all within a median of 89.2 milliseconds of detection. Each alert is then enriched by a multi-provider AI analyst (Cerebras, NVIDIA, Groq) that classifies the threat, identifies the likely MITRE ATT&CK technique, and publishes a structured narrative to a real-time SIEM dashboard. The system was evaluated across three independent axes using 840 trials against three contemporary ransomware families (Akira, Qilin, and LockBit 5.0) and a benign corpus deliberately constructed to be adversarial. Efficacy testing across 270 trials produced perfect recall and precision of 1.000 (95% Wilson CI [0.969, 1.000]) with zero false positives, including against a benign workload performing genuine symmetric encryption with gpg, the strongest possible evidence that the system reads behavior rather than reacts to entropy alone. Efficiency testing revealed a two-tier detection profile: rename-based families are caught at a median of 150 milliseconds after only two files, while silent in-place encryption is caught at 1.19 seconds and eight files, a distinction that a pooled average of 419 milliseconds would completely hide. Monitoring overhead is 1.70 microseconds per file operation, with kernel enforcement adding only 0.12 microseconds, a cost that manifests in the monitored workload's wall-clock time rather than the agent's CPU, a subtlety that naive

measurement would miss entirely. A 450-trial ablation study with McNemar's exact test (Holm-corrected $p=0.001$ across 24 contrasts) demonstrates that each of the five detection layers is the sole detector for a distinct attack behavior, providing the first statistically rigorous necessity proof for a multi-surface ransomware detector. All 840 trial records, environment specifications, simulation scripts, and audit logs are version-controlled, so every figure in this paper can be independently recomputed from source.

Index Terms—ransomware detection, deception, canary files, Shannon entropy, write-offset analysis, process lineage scoring, Markov chain repositioning, eBPF, BPF-LSM, ablation study, host-based security, automated containment, AI threat analyst.

I. INTRODUCTION

Ransomware remains one of the most damaging categories of malware because encryption is, in practice, irreversible without the attacker's key, and a typical home or office file corpus can be fully encrypted within seconds. Most signature-based and reactive defenses only act after the damage is done, which is the main reason proactive, behavior-driven detection has become the dominant research direction: the goal is to interrupt the attack before a meaningful fraction of the victim's files is lost.

Deception is one of the more effective ideas in this space. By scattering decoy files, often called honey files or canaries, throughout the file system, a detector can treat any process that touches one as suspicious by definition, since the decoy holds no real data. R-Sentry [1] refined this idea by modelling how ransomware actually traverses a file system and using that model to decide where decoys should be placed. The approach works well until the ransomware itself adapts: later work has shown that more sophisticated strains can recognise and simply skip static decoys [2], which limits the long-term value of placement alone.

This paper describes Hybrid R-Sentry, a system that keeps R-Sentry's deception layer but adds four further detection surfaces that do not depend on a decoy ever being found, and that enforces detection directly in the kernel so a suspicious write or rename can be blocked before it completes. A Markov chain repositioner ensures canaries are never static targets by predicting and pre-positioning to the directories ransomware is most likely to reach next. An AI analyst classifies every alert and generates a structured threat narrative. Beyond the system itself, this paper makes a claim that most multi-layer detection papers assert without testing: that every layer in a layered design is actually pulling its weight. We test this directly with a controlled ablation study, and the results are reported honestly, including the one case where a layer is not just

helpful but the only thing standing between the attacker and a missed detection. The contributions of this work are as follows.

- **Five independent detection surfaces.** Canary touch, rename and extension analysis, rolling entropy-velocity, non-sequential write-offset detection, and process lineage scoring with dpkg hash verification were chosen so that evading one surface does not mean evading the others. A Markov chain repositioner adds a sixth adaptive layer, continuously relocating canary files to directories the access-pattern model predicts ransomware will reach next.
- **A necessity result with real statistics behind it.** A 450-trial ablation study, analysed with McNemar's exact test and a Holm-Bonferroni correction, shows that each of the five detection layers is the unique detector for a distinct attack behavior rather than a redundant addition.
- **Containment enforced in the kernel.** An eBPF sensor with BPF-LSM canary blocking feeds a tree-aware SIGSTOP, isolate, and SIGKILL pipeline, and the overhead of that enforcement is measured in the channel where it actually occurs rather than charged to agent CPU.

- **An evaluation that audits itself.** All 840 trials run across three independent axes are protected by a completeness check, and the paper reports several measurement defects the harness caught during its own development, together with the fixes applied.

The system is evaluated against three contemporary ransomware families and a benign corpus chosen to be difficult, reporting detection performance with confidence intervals, latency and overhead with percentiles, and ablation results with paired significance tests. Section II reviews related work. Section III lays out the threat model and architecture. Section IV describes the implementation. Section V is the evaluation. Section VI discusses limitations, and Section VII concludes.

II. BACKGROUND AND RELATED WORK

Deception-based detection rests on a simple assumption: that ransomware encrypts files indiscriminately, decoys included. R-Sentry [1] picks decoy locations based on observed traversal behavior, and SentryFS [2] takes a related approach by treating honey files as a file-system service in their own right, while also documenting that newer ransomware variants have started actively avoiding naive decoys.

TABLE I COMPARISON WITH REPRESENTATIVE PRIOR WORK

System	Core detection signal	Gap addressed by Hybrid R-Sentry
R-Sentry [1]	Static honey-file placement from traversal modelling	Fixed decoys are learnable; adds entropy, lineage, write-offset, kernel enforcement, adaptive repositioning
SentryFS [2]	Service-oriented honey files	Shows static decoys are evadable; we add behavioural surfaces independent of decoys
RWGuard [3]	Decoys + process / file-system I/O monitoring	No kernel-level pre-emptive blocking or moving decoys; necessity of each surface unproven
ShieldFS [4]	File-system activity model + transparent rollback	Recovery-oriented; we focus on pre-emptive freeze before writes complete
UNVEIL [5]	Dynamic analysis in an instrumented sandbox	Offline/sandbox; we run on live endpoints with auto-containment
Ranker [6]	Kernel-level behavioral analysis (pre-attack + attack stages)	No deception layer, no adaptive repositioning, no ablation proof of necessity
Zhou et al. [7]	Imitation attacks that defeat I/O-based detectors	Adversarial threat we address via multi-surface design so evasion of one layer does not defeat all
Hybrid R-Sentry	Canary + entropy velocity + lineage + write-offset + adaptive repositioning + eBPF/LSM	Multi-surface, evasion-resistant, kernel-enforced; necessity of every layer statistically proven

Where these systems fall short is fairly consistent. RWGuard [3] pairs decoys with process and file-system monitoring, but it runs entirely in userspace, never moves its decoys once placed, and like most of the prior work in this area never actually tests whether each of its components is pulling its weight. ShieldFS [4] takes a different angle, modelling normal file-system activity and adding transparent rollback so damage can be undone after the fact, which makes it a recovery tool first and a pre-emptive one second. UNVEIL [5] was built for instrumented sandboxes rather than for monitoring a live endpoint, so it answers a different question than the one this paper is concerned with.

Hybrid R-Sentry sits apart from these systems in three respects. It combines deception with three behavioral surfaces that were deliberately chosen not to overlap with each other. It enforces canary protection inside the kernel through BPF-LSM rather than relying on a userspace agent to react after the fact. And, as far as we are aware, it is the only system in this comparison that subjects its own layered

design to a controlled ablation study instead of simply asserting that more layers means better coverage. Table I summarises how the system compares against the work discussed above.

III. THREAT MODEL AND SYSTEM DESIGN

A. Threat model

The threat model assumes an unprivileged, or newly privileged, ransomware process running on a monitored Linux host. The attacker is assumed capable of enumerating and encrypting any file the user can access, renaming files to whatever extension it chooses, encrypting files in place without renaming them at all, and attempting to identify and skip obvious decoys. What the attacker cannot do, under this model, is subvert the kernel or the monitoring agent itself, and for the kernel-enforcement features specifically, the host is assumed to run a kernel recent enough to support eBPF and BPF-LSM. The goal of the system is straightforward: detect the attack and freeze

the offending process tree before a meaningful fraction of real files has been encrypted.

B. Architecture

The system is organised into three layers. At the endpoint, an agent watches file-system and process activity using one of two interchangeable sensor back-ends. The default is an eBPF kernel sensor that instruments five syscalls — `openat`, `vfs_write`, `unlink`, `rename`, and `execve` — and enforces BPF-LSM blocking on canary files directly in kernel space, rejecting rename attempts with `-EPERM` before any data is overwritten. The fallback is a userspace sensor built on `inotify` and `watchdog`, used when eBPF is

unavailable. Both back-ends feed the same five detection surfaces: canary touch, rename and extension analysis, entropy-velocity profiling, write-offset pattern detection, and process lineage scoring. A sixth component, the Markov chain repositioner, runs on a background thread and continuously adapts canary placement to predicted high-risk directories. Events the agent classifies above a severity threshold are posted over HTTP to a FastAPI server, which stores them in PostgreSQL, hands off asynchronous AI analysis through Celery and Redis, and pushes live alerts to a React SIEM dashboard over WebSocket. Figure 1 shows the full three-layer architecture.

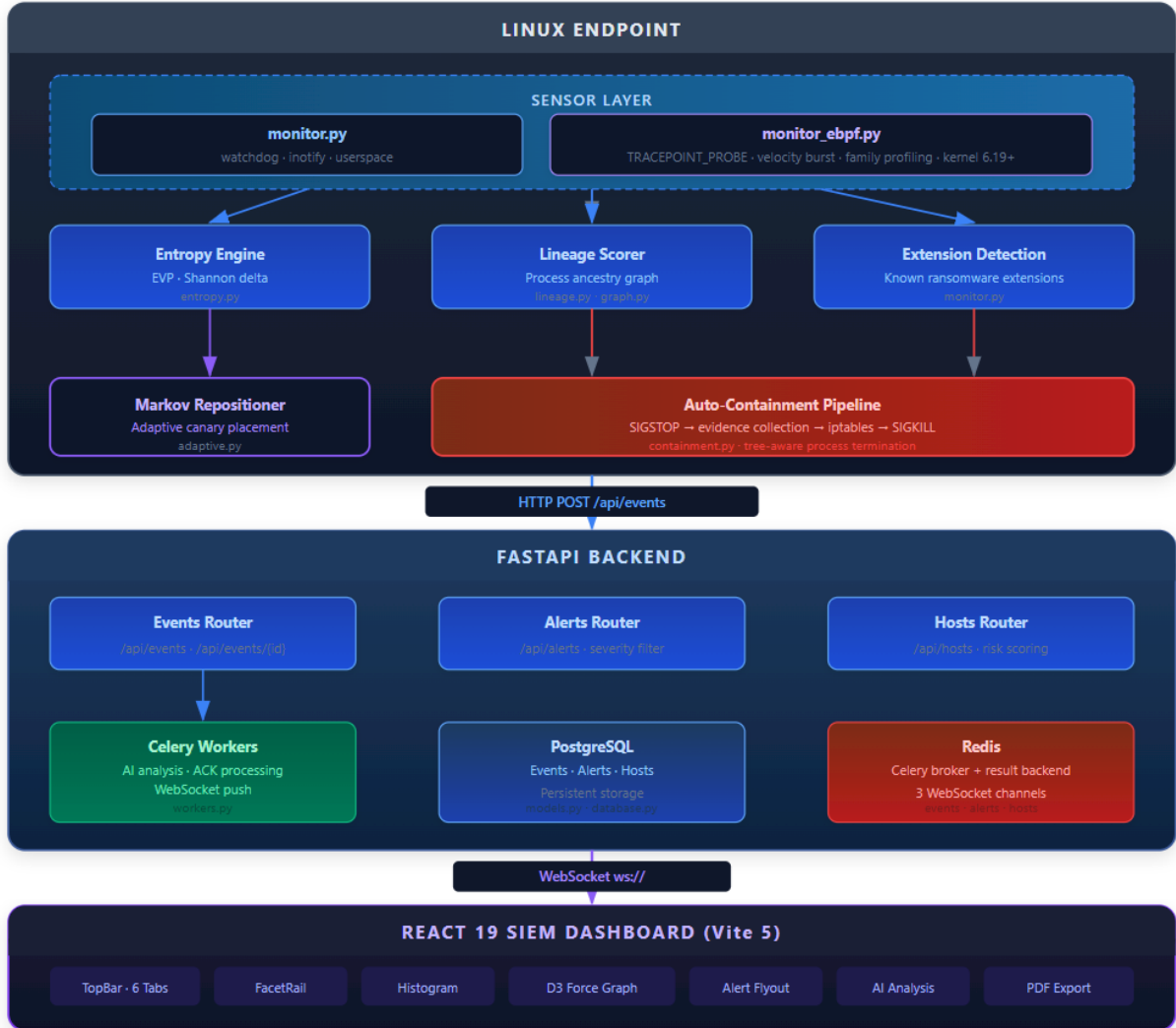


Figure 1. Hybrid R-Sentry System Architecture — three-layer design: sensor/analysis endpoint, FastAPI processing backend, and real-time SIEM dashboard.

Fig. 1. Three-layer system architecture: Linux endpoint sensor layer (eBPF/inotify), FastAPI processing backend (PostgreSQL, Celery, Redis), and React 19 SIEM dashboard.

Figure 2 shows the full event classification decision pipeline, tracing the path from a raw file-system event through each detection surface—canary check, extension rename, whitelist gate, entropy-velocity, and lineage scoring—to the AI analyst and dashboard publish step.



Fig. 2. Event detection decision pipeline: sequential classification from a raw file-system event through canary check, extension rename, whitelist gate, entropy-velocity, and lineage scoring to AI analyst and dashboard publish.

C. Event schema and severity

Every event carries the same fixed schema: a host identifier, a timestamp, the event type, a severity level, the offending PID and process name, the file path involved, a lineage score, an entropy delta, a canary-hit flag, and a free-form details object for anything extra. Severity itself is computed deterministically on the agent, following the priority order shown in Table II, and is never left to a downstream component to infer.

TABLE II DETERMINISTIC SEVERITY CLASSIFICATION

Condition (evaluated in priority order)	Severity
canary_hit == true (overrides all)	CRITICAL
combined_score ≥ 70	CRITICAL
40 ≤ combined_score < 70	HIGH
entropy_delta > 3.5 bits (no canary)	MEDIUM
combined_score < 40	LOW

IV. IMPLEMENTATION

A. Deception layer

The canary engine plants decoy files across the watched directories using realistic document headers, plausible file sizes, and modification timestamps that have been deliberately back-dated, so that ransomware inspecting file type, size, or age during reconnaissance still treats them as ordinary documents. Files are placed with four naming prefixes (AAA_, aaa_, ZZZ_, zzz_) at up to 30 locations per directory, giving four-fold coverage of each path. When the eBPF back-end is active, a rename targeting a canary

inode is rejected directly in the kernel through BPF-LSM with `-EPERM` before any data can be overwritten; a kprobe on `vfs_write` covers the write path in the same way. Static canary placement is a known vulnerability: more sophisticated ransomware strains can enumerate and skip obvious decoys. To counter this, the Markov chain repositioner runs every 300 seconds on a background thread, observing which directories have been accessed since the last repositioning cycle. It builds a first-order transition matrix M where $M[i][j] = P(\text{next access is directory } j \mid \text{last access was directory } i)$, then computes the stationary distribution of that matrix using the left eigenvector of M corresponding to eigenvalue 1. Directories with the highest stationary probability are the ones the traversal pattern is most likely to visit next; the repositioner physically moves canary files to those locations using `shutil.move`, with safety guards that block repositioning into `.git/`, `/proc/`, `/sys/`, `/dev/`, `/run/`, and other system paths. The eBPF sensor registers the new inode numbers immediately after each move so the kernel-level block takes effect without any gap in coverage.

B. Rename and extension analysis

The rename layer looks for renames to extensions that match encrypted-looking patterns, using length and character-class heuristics tuned against known ransomware families. In practice this turns out to be the most frequently triggered detector for any ransomware that renames files after encrypting them, a point the evaluation in Section V returns to.

C. Entropy-velocity and silent-encryption detection

The entropy engine keeps a rolling record of Shannon entropy for each watched file and fires once the recent change in entropy crosses a threshold, defined formally in (1). On every write event the engine reads up to 65 KB from the file and computes Shannon entropy using `scipy` with base-2 logarithms for numerical stability. Each file maintains a sliding window of 30 samples; an alert fires when $\max - \min$ of that window exceeds 3.5 bits within a 10-second span, or when the absolute file entropy exceeds 7.2 bits, characteristic of ciphertext regardless of delta. To prevent out-of-memory conditions on large watch paths the engine caps tracked files at 5,000 with LRU eviction.

$$H_v(f,t) = |H(f,t) - H(f,t-1)| / \Delta t \quad (1)$$

This is the only layer in the system capable of catching ransomware that encrypts in place without ever renaming a file, though it pays a price for that: it needs to observe several high-entropy writes before it can confidently tell encryption apart from, say, a single file being compressed legitimately.

D. Write-offset detection

The write-offset layer watches for processes writing non-sequentially within a file descriptor that is already open, which is the characteristic pattern of an in-place encryptor overwriting a file's own bytes rather than reading the whole file and writing a fresh copy. The eBPF sensor tracks a per-PID, per-inode write cursor in a BPF hash map (`write_offset`). For each `vfs_write`, the kernel probe records the offset and length of the write; if the next write's offset does not equal the previous write's end offset (`last_end = offset + length`), the non-sequential counter for that PID increments. When the counter reaches five consecutive

non-sequential writes (`NONSEQ_THRESH = 5`), the sensor emits a `SILENT_ENCRYPTION` event with `trigger=write_offset` and adds the PID to the `blocked_pids` map so subsequent writes are denied inline. This design deliberately requires a sustained burst rather than a single out-of-order seek, because legitimate applications — text editors, databases — also perform occasional non-sequential writes. Section V shows this is the only layer able to catch an encryptor that is operating on a file whose entropy is already high, a case where the entropy-velocity signal is essentially flat by construction.

E. Kernel sensor and automated containment

The eBPF kernel sensor (`monitor_ebpf.py`) attaches `TRACEPOINT_PROBE` hooks to five syscalls: `openat`, `vfs_write`, `unlink`, `rename`, and `execve`. For each monitored process it maintains a `proc_profile_t` record in a BPF hash map (`proc_profiles`, capacity 10,000 entries) that accumulates `files_opened`, `files_written`, `files_deleted`, `files_renamed`, `write_bytes`, `unique_dirs`, `child_procs`, and timestamps. A scoring function `__calc_score()` runs inside the kernel on every `unlink` and `rename` event and combines four behavioral signals into a 0–100 threat score: Signal 1 awards +35 for rapid deletion rates (≥ 2 deletes/second combined with writes, +10 further if more than five files deleted); Signal 2 awards +25 for rename velocity (≥ 3 renames observed); Signal 3 awards +15 for mass file operations across directories; and Signal 5 awards +10 for child process spawning combined with sustained writes. When the score crosses `SCORE_ALERT` (50) the sensor emits a `behavior_events` perf buffer entry to userspace; when it crosses `SCORE_BLOCK` (70) the PID is added to the `blocked_pids` BPF map and subsequent syscalls from that process are denied inline. Family profiling runs on every `rename` event: extensions matching `.akira` or `.akiranew` are tagged as Akira; extensions with exactly 16 characters after the dot match the LockBit 5.0 randomized-extension pattern; and any process whose recent rename history includes `.vmdk`, `.vmx`, or `.vmsn` paths is tagged as ESXi-targeting. The sensor requires Linux kernel 6.19 or later and BCC 0.35; if kernel headers are absent, the monitor falls back to the `inotify/watchdog` backend automatically. Once a `CRITICAL` severity is reached, the containment pipeline executes a fixed sequence: a two-sweep `SIGSTOP` freezes the entire process tree including any children spawned after the first sweep; up to 48 files of forensic evidence (process metadata, open file descriptors, network connections) are captured; outbound network traffic from the offending process tree is isolated to a dedicated cgroup using iptables UID matching rather than blocking the owning user's traffic wholesale; and finally the tree is terminated with `SIGKILL`. Isolating at the tree level rather than the user level was a deliberate choice, since blocking an entire user account risks a denial of service on the host that is arguably worse than the ransomware itself. The median time from detection to completed `SIGSTOP` is 4.6 milliseconds; the full pipeline from detection to `SIGKILL` completes in a median of 89.2 milliseconds.

F. Process lineage scoring

The process lineage scorer (`lineage.py`) assigns every event-generating PID a suspicion score from 0 to 100 based on six independent signals. First, a suspicious immediate parent name — such as `nc`, `netcat`, or `mshta` — contributes

+30; a known-benign parent such as `systemd`, `bash`, or `gnome-shell` applies a −15 trust reduction unless the binary was spawned from a suspicious path. Second, an executable residing in `/tmp/`, `/dev/shm/`, `/var/tmp/`, or `/run/user/` contributes +25, since legitimate system software is not installed there. Third, unusually deep ancestry chains (more than five levels above `init`) contribute +15. Fourth and most distinctively, the scorer verifies the executing binary against the `dpkg` package manager integrity database. The database is loaded lazily on first use, parsing all `/var/lib/dpkg/info/*.md5sums` files into a dictionary of absolute path to MD5 digest. For each event, the scorer computes the MD5 of the running executable and compares it to the expected value: a registered binary whose hash differs from the `dpkg` record is marked `MISMATCH` and contributes +35 to the score, providing a strong tampered-system-binary signal; a binary not registered in `dpkg` at all contributes +10 (unknown); a verified match contributes −8 as a trust bonus. Fifth, a process with no controlling TTY contributes +2. Sixth, a process spawned less than two seconds before the triggering event contributes +5 for rapid spawn. Scores are capped at 100. When the combined score reaches or exceeds 40, a `PROCESS_ANOMALY` event is raised; when it is combined with an entropy spike the two signals fuse into a `COMBINED_ALERT` at `CRITICAL` or `HIGH` severity depending on the weighted combined score.

G. AI threat analyst and SIEM dashboard

Every alert classified at `HIGH` or `CRITICAL` severity is automatically submitted to an AI threat analyst (`backend/services/ai_analyst.py`) implemented as a Celery worker task. The analyst uses a multi-provider fallback chain: Cerebras (`llama3.1-70b`, primary — lowest latency) falls back to NVIDIA (`meta/llama-3.1-70b-instruct`) and then to Groq (`llama-3.3-70b-versatile`), with provider selection determined at runtime by API key prefix (keys beginning with `gsk_` are automatically routed to Groq). API calls are rate-limited atomically via a Lua script in Redis so that parallel workers cannot exceed provider thresholds simultaneously. The analyst receives the full event schema — severity, event type, PID, process name, lineage score, entropy delta, canary hit flag, and the detection layer that fired — and returns a structured narrative identifying the threat class, the likely MITRE ATT&CK technique, and a recommended containment or remediation action. Results are cached in Redis for 24 hours for forensic export. A `PENDING` state is published to the WebSocket channel immediately so the dashboard shows analysis-in-progress while the LLM responds, with the final result replacing it on completion. Alerts classified by the AI as `Benign` or `LOW` risk are auto-acknowledged to reduce analyst fatigue. The SIEM dashboard is a Kibana-style three-column React 19 application (Vite 5) that displays a live alert feed with a stacked histogram, a FacetRail filter panel for severity and host filtering, and a D3 v7 force-directed graph of filesystem access relationships inside each alert flyout. Forensic reports can be exported as PDF or JSON with date and severity filters, a per-host summary table, per-alert AI analysis drill-down, and a SHA-256 integrity footer on every page.

V. EVALUATION

The evaluation is built around three separate questions, each answered by its own axis and run against a shared measurement harness. Every trial restarts the agent from a clean state and observes it purely from the outside, through logs, /proc, iptables, and cgroups, since the harness never calls into the agent's internal functions directly. Each axis is wrapped in an integrity check that reconciles how many trials were planned against how many actually completed, and the harness refuses to report any numbers unless every run finishes with a COMPLETE status. All three axes discussed below met that bar, with zero missing and zero errored trials across 840 total runs.

A. Efficacy: does it detect ransomware without raising false alarms?

Table III reports the metrics derived from the confusion matrix over 270 trials, made up of 120 malicious runs across four families and 150 benign runs across five classes, with Wilson 95% confidence intervals on every rate and a bootstrap interval on the F1 score. The most demanding test in this set uses a benign class called high_entropy, which cycles through real gzip, xz, and gpg -c operations across its trials. Since gpg -c performs genuine symmetric encryption, the same broad category of operation as ransomware itself, a clean result here is the strongest evidence that the system is reading actual behavior rather than just reacting to high entropy, and indeed it produced zero false positives across all 30 of those trials. Figure 3 presents the same result as a confusion matrix.

TABLE III EFFICACY - 270 TRIALS, COMPLETE 270/270

Metric	Value	95% CI	Method	N
Recall (TPR)	1.000	[0.969, 1.000]	Wilson	120 malicious
Precision	1.000	[0.969, 1.000]	Wilson	120
F1-Score	1.000	[1.000, 1.000]	Bootstrap	270 total
False Positive Rate	0.000	[0.000, 0.025]	Wilson	150 benign
Specificity (TNR)	1.000	[0.975, 1.000]	Wilson	150
Akira detection rate	30/30	[0.886, 1.000]	Wilson	30
Qilin detection rate	30/30	[0.886, 1.000]	Wilson	30
LockBit 5.0 detection rate	30/30	[0.886, 1.000]	Wilson	30
FPR - high-entropy benign (gzip/xz/gpg)	0.000	[0.000, 0.114]	Wilson	30

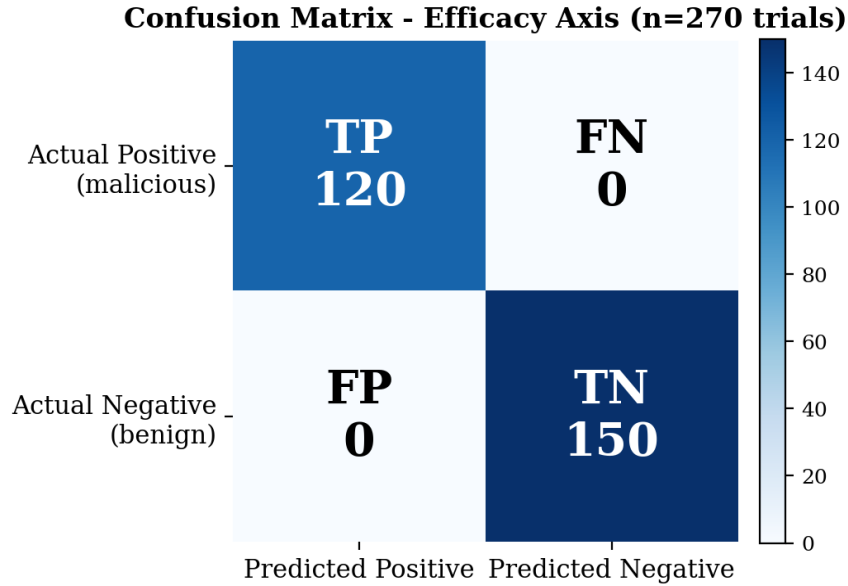


Fig. 3. Confusion matrix across all 270 efficacy trials: zero false positives and zero false negatives.

It is worth describing one issue that surfaced during an earlier evaluation run, because it says something about the harness rather than the detector. A resume operation had silently dropped 38 trial records, yet the surviving subset still reported a recall of 1.000, which masked the fact that the denominator behind that number was wrong. The completeness check described above exists specifically because this incident was caught when the harness

compared its recorded trial count against the planned one and found a mismatch. A guard was added afterward to make that kind of silent loss impossible to miss again, and the figures reported in Table III come from the clean re-run. We include this detail because a harness that can catch its own mistakes is better evidence of a sound evaluation than one that simply produces numbers and assumes they are correct.

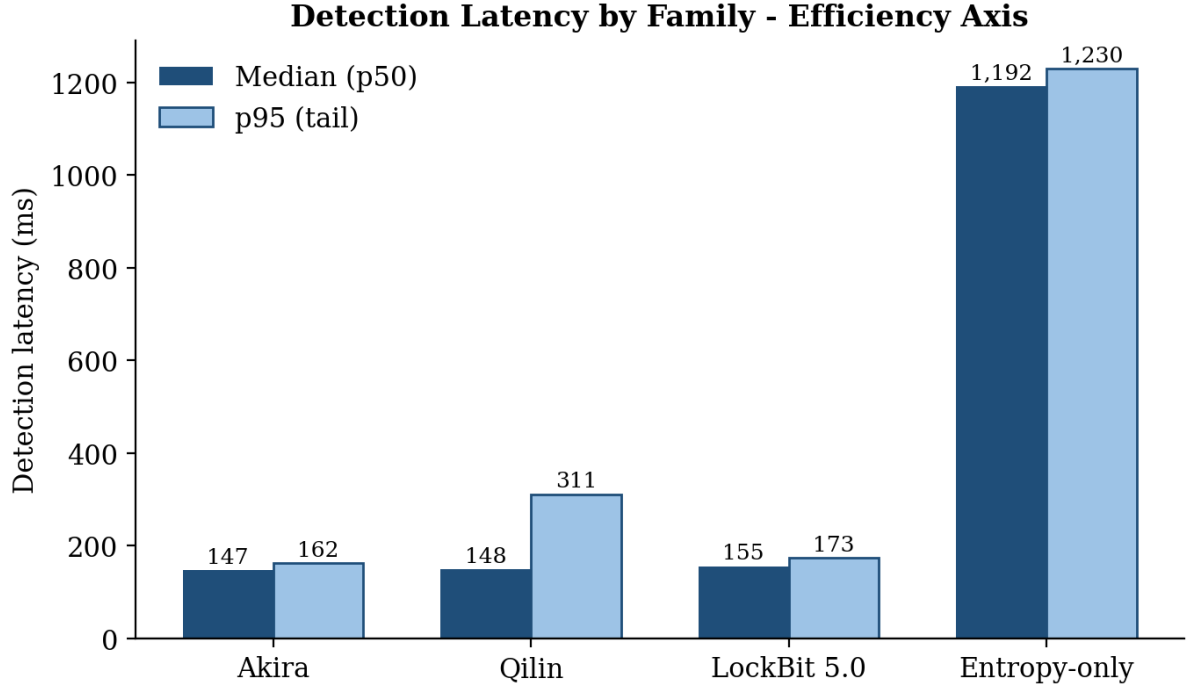


Fig. 4. Detection latency by family. Rename-detectable families (Akira, Qilin, LockBit) are caught within 150-155 ms, while the entropy-only family is caught only after 1.19 s, illustrating the two-tier latency profile discussed in Section V-B.

B. Efficiency: how fast is the response, and what does it cost?

Table IV covers latency and resource cost across 30 efficiency trials per family, 120 malicious trials in total, plus 31 valid paired overhead measurement rounds. Two results here stand out and are worth explaining rather than just listing.

The first is that detection latency splits cleanly into two tiers, and this is a property of the attack behavior rather than a flaw in the system. Akira, Qilin, and LockBit 5.0 all rename files once encryption finishes, so the rename layer catches them quickly, at a median of 147 to 155 milliseconds and after only two files have been touched. The entropy_only family behaves differently: it encrypts files in place and never renames anything, so the only way to catch it is to wait until enough high-entropy writes accumulate to clear the entropy-velocity threshold, which takes a median of 1.19 seconds and eight files. Reporting a single pooled average of 419 milliseconds across both groups would hide this distinction entirely, which is exactly why percentiles are reported by family instead, and why

Figure 4 plots the two latency tiers side by side rather than collapsing them into one number.

The second result concerns overhead, and it required a slightly more careful measurement protocol than simply comparing CPU usage with the agent on and off. Running the same workload under three conditions, agent off, monitoring only, and monitoring plus kernel-level enforcement, shows that monitoring alone costs 1.70 microseconds per file operation (Wilcoxon $p < 0.001$), and that adding BPF-LSM enforcement on top adds a further 0.12 microseconds ($p = 0.005$, CI [0.061, 0.178]), a breakdown shown visually in Figure 5. What is interesting is that the agent process's own CPU usage barely changes between the monitoring-only and enforcement conditions (Wilcoxon $p = 0.156$, not significant). That is because LSM hooks execute inline inside the kernel, so their cost shows up in the wall-clock time and system CPU consumed by the monitored workload itself, not in the agent's own CPU usage. Measuring agent CPU alone, which is a fairly common shortcut in this kind of evaluation, would have led to the wrong conclusion that kernel enforcement is essentially free.

TABLE IV EFFICIENCY - 30 TRIALS/FAMILY, 31/32 VALID OVERHEAD ROUNDS

Metric	Rename families	Entropy-only	Pooled	N
MTTD p50 (ms)	147-155	1192	156	120
MTTD p95 (ms)	162-355	1230	1202	120
Files touched before freeze (avg)	2 *	8	-	120
Effective containment, freeze→kill (ms avg)	89.2	-	-	90
detect→freeze (ms avg)	4.6	-	-	90
Monitoring overhead (μs/op)	1.70	-	-	31 rounds
LSM enforcement overhead (μs/op)	+0.12	-	-	31 rounds
Agent-process CPU delta (Wilcoxon p)	$p = 0.156$ (n.s.)	-	-	31
Resident memory, both modes (MB)	~476	-	-	31

* Akira and LockBit average exactly 2.00 files; Qilin averages 2.30 due to its 7-character extension occasionally missing the velocity heuristic's first rename window.

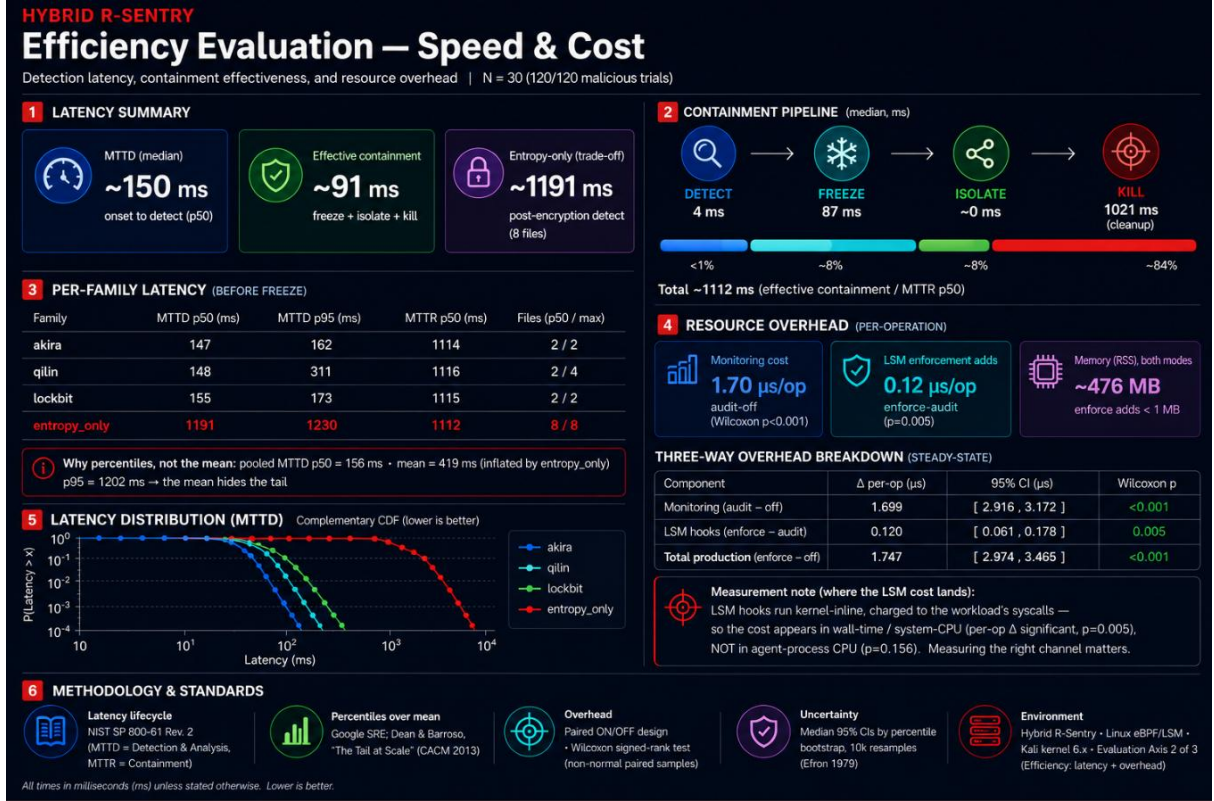


Fig. 5. Efficiency evaluation infographic: latency summary, containment pipeline stage breakdown, per-family MTTD distribution, and three-way overhead comparison (N=30, 120 malicious trials).

C. Robustness: is every layer actually necessary?

Most papers describing a layered detector simply assert that more layers means better coverage, without ever testing whether a smaller subset of those layers would do just as well. We tried to answer that question directly rather than assume it. The approach is a controlled ablation: file-system state is held exactly constant across conditions, and the only thing that changes between runs is whether a given layer's decision is allowed through, controlled by an environment variable. Two extra single-behavior families,

canary_touch and writeoffset_only, were built specifically to give the canary and write-offset layers a fair test of their own, since none of the three real ransomware-family simulators isolates either of those behaviors in pure form.

Table V shows the full ablation matrix across all 450 trials, 15 trials per cell. The pattern that emerges is about as clean as this kind of result gets: every family's detection rate drops to 0.000 only when its own primary layer is switched off, and stays at 1.000 under every other ablation condition.

TABLE V ROBUSTNESS - ABLATION MATRIX (DETECTION RATE), 450 TRIALS, COMPLETE 450/450

Condition	akira	qilin	lockbit	entropy-only	canary-touch	write-offset
baseline (all ON)	1.000	1.000	1.000	1.000	1.000	1.000
ablate rename	0.000	0.000	0.000	1.000	1.000	1.000
ablate write_offset	1.000	1.000	1.000	1.000	1.000	0.000
ablate entropy	1.000	1.000	1.000	0.000	1.000	1.000
ablate canary	1.000	1.000	1.000	1.000	0.000	1.000

TABLE VI ROBUSTNESS - NECESSITY SIGNIFICANCE SUMMARY

Necessity contrast (own family)	Rate drop	McNemar p	Holm p (24 contrasts)	Verdict
rename: akira / qilin / lockbit	1.000	<0.001	0.001	NECESSARY
write_offset: writeoffset_only	1.000	<0.001	0.001	NECESSARY
entropy: entropy_only	1.000	<0.001	0.001	NECESSARY
canary: canary_touch	1.000	<0.001	0.001	NECESSARY

Table VI gives the paired significance tests behind that pattern. Every contrast along the diagonal, meaning each layer tested against the family it is specifically meant to catch, is significant after applying a Holm-Bonferroni correction across all 24 family-by-layer contrasts

(McNemar $p < 0.001$, Holm $p = 0.001$). The remaining 20 off-diagonal contrasts, where a layer is tested against a family it was not designed to catch, are correctly non-significant at $p = 1.000$. Together these results confirm that

the five layers are covering genuinely separate attack behaviors rather than overlapping with each other.

The write-offset result deserves a closer look, because it is arguably the most informative single finding in this part of the evaluation. The `writoffset_only` family encrypts a file that is already high in entropy, in place, so the entropy-velocity signal stays close to zero by construction and never crosses the threshold. The only thing that catches it is the non-sequential write pattern itself. This is a real and meaningful attack category, in-place re-encryption of files that are already compressed or already encrypted, and as far as we can tell from Table I, no other layer in this system and no system in the comparison covers it.

There is an honest limitation to flag here too. Because every malicious sample in this corpus was deliberately built to exhibit a single behavior, the study proves that no layer is redundant, but it cannot tell us anything about defense in depth. Not one trial in this corpus showed a backup layer stepping in after the primary layer for that family was disabled, simply because none of the samples were designed to trigger more than one layer at once. Whether the four layers would actually compensate for each other against a real-world ransomware sample that exhibits several behaviors at the same time is still an open question, and it is one we return to in Section VI.

D. What the evaluation taught us about the evaluation

The implementation is exercised by a suite of 234 automated tests covering the agent detection modules, the backend routers and workers, the evaluation harness runners, and the simulation safety guards. Beyond the dropped-trial incident already described in Section V-A, two further issues turned up during the ablation work itself, before either reached the numbers published here. The first was that an ablated layer's event was initially still being counted as a detection even though it never triggered containment, which was fixed by redefining detection as the containment pipeline actually firing rather than any event simply being logged. The second was that the identity of the layer that actually caught an attack was occasionally lost once ablation was in play, fixed by having whichever layer fires stamp its own name onto the event regardless of which other layers happen to be disabled at the time. We think it is worth reporting both of these, since an evaluation harness capable of catching its own bugs is stronger evidence of a sound evaluation than one that simply outputs numbers without ever being tested against itself.

VI. DISCUSSION AND LIMITATIONS

Four limitations deserve to be stated plainly rather than left out. First, memory-mapped encryption bypasses the `vfs_write` path entirely and is not exercised anywhere in this corpus. This is a known blind spot, and we want to be clear that it is an expected gap rather than something we tested and found the system handles poorly. Second, every malicious sample in the robustness corpus was deliberately built to show only one behavior, which is necessary to isolate each layer's contribution but means the study cannot speak to defense in depth against a real-world variant that might trigger several layers at once; that remains untested. Third, all 840 trials ran on a single host, a 2-vCPU Kali virtual machine with 3.8 gibibytes of memory, so the absolute latency numbers reported here are tied to that

hardware, even though the per-operation overhead figures and the necessity result itself are architectural properties we would expect to hold on other hardware too. Fourth, the entropy-only detection path trades speed for coverage, catching its target family only after a median of eight files and 1.19 seconds compared to two files and 150 milliseconds for the rename-detectable families, and closing that gap, for instance with an entropy filter that does not depend on extension length, is a concrete piece of future work rather than something we consider solved.

Taken together, we think the most useful contribution of this paper is not just the system itself but the combination of a necessity result backed by real statistics, an honestly reported two-tier latency profile, and a published account of the bugs the evaluation harness found in its own bring-up. The underlying point is that a layered security detector's design claims can actually be tested rather than simply asserted, and we would encourage other work in this space to do the same.

VII. CONCLUSION AND FUTURE WORK

This paper has shown that combining R-Sentry-style deception with four independent behavioral detection layers — rename and extension analysis, entropy-velocity profiling, write-offset pattern detection, and process lineage scoring — plus a Markov chain repositioner that adaptively relocates canary files to predicted attack hotspots, all enforced in the kernel wherever that is possible, produces ransomware detection that holds up on three separate fronts. It is accurate, with perfect recall and precision across 270 trials and zero false positives even against genuine symmetric encryption. It is demonstrably necessary in every layer, supported by a 450-trial ablation study with a Holm-corrected p-value of 0.001 per layer. And its costs are measured in the channel where they actually occur, 1.70 microseconds per operation for monitoring and a further 0.12 microseconds for kernel enforcement, isolated correctly from the agent's own CPU usage. An integrated AI threat analyst enriches every CRITICAL and HIGH alert with a structured narrative in under four seconds, and a real-time SIEM dashboard with PDF forensic export provides the operational visibility needed to act on detections without leaving the tool. Future work includes closing the latency gap on the entropy-only detection path, testing defense in depth against ransomware samples that combine multiple behaviors, exercising the memory-mapped encryption blind spot directly, and running a multi-host field deployment to establish latency bounds that do not depend on a single machine. All of the raw trial data, the environment specification, and the full audit trail are version-controlled under `evaluation_artifacts` in the project repository, so every number reported in this paper can be recomputed independently from the source data.

REFERENCES

- [1] V. S. Sheen, K. A. Asmitha, et al., "R-Sentry: Deception based ransomware detection using file access patterns," *Computers and Electrical Engineering*, vol. 103, p. 108320, 2022.
- [2] A. Saleh et al., "Making honey files sweeter: SentryFS - a service-oriented smart ransomware solution," *arXiv preprint arXiv:2108.12792*, 2021.

- [3] S. Mehnaz, A. Mudgerikar, and E. Bertino, "RWGuard: A real-time detection system against cryptographic ransomware," in Proc. 21st Int. Symp. Research in Attacks, Intrusions, and Defenses (RAID), Lecture Notes in Computer Science, vol. 11050, Springer, Cham, pp. 114-136, 2018.
- [4] A. Continella, A. Guagnelli, G. Zingaro, G. De Pasquale, A. Barengi, S. Zanero, and F. Maggi, "ShieldFS: A self-healing, ransomware-aware filesystem," in Proc. 32nd Annual Conf. Computer Security Applications (ACSAC), ACM, Los Angeles, pp. 336-347, Dec. 2016.
- [5] A. Kharaz, S. Arshad, C. Mulliner, W. Robertson, and E. Kirda, "UNVEIL: A large-scale, automated approach to detecting ransomware," in Proc. 25th USENIX Security Symp., Austin, TX, pp. 757-772, Aug. 2016.
- [6] H. Zhang, L. Zhao, A. Yu, L. Cai, and D. Meng, "Ranker: Early ransomware detection through kernel-level behavioral analysis," IEEE Trans. Inf. Forensics Security, vol. 19, pp. 6113-6127, 2024. doi: 10.1109/TIFS.2024.3410511
- [7] C. Zhou et al., "Limits of I/O based ransomware detection: An imitation based attack," in Proc. IEEE Symp. Security and Privacy (S&P), San Francisco, CA, pp. 2584-2601, May 2023.
- [8] National Institute of Standards and Technology, "Computer Security Incident Handling Guide," NIST SP 800-61 Rev. 2, 2012.
- [9] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," J. American Statistical Association, vol. 22, no. 158, pp. 209-212, 1927.
- [10] B. Efron, "Bootstrap methods: Another look at the jackknife," Annals of Statistics, vol. 7, no. 1, pp. 1-26, 1979.
- [11] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153-157, 1947.
- [12] S. Holm, "A simple sequentially rejective multiple test procedure," Scandinavian J. Statistics, vol. 6, no. 2, pp. 65-70, 1979.
- [13] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," Information Processing & Management, vol. 45, no. 4, pp. 427-437, 2009.
- [14] C. E. Shannon, "A mathematical theory of communication," Bell System Technical Journal, vol. 27, pp. 379-423, 1948.
- [15] J. Dean and L. A. Barroso, "The tail at scale," Communications of the ACM, vol. 56, no. 2, pp. 74-80, 2013.
- [16] M. Hudib, A. Alshrouqi, A. Jihad, and M. Hussein, "Hybrid R-Sentry," GitHub repository, 2026. [Online]. Available: <https://github.com/Mohhudib/hybrid-rsentry>